

C++ Standard Library Issues to be moved in Albuquerque

Doc. no. R0815R0

Date: Revised 2017-10-16 at 05:08:47 UTC

Project: Programming Language C++

Reply to: Marshall Clow <lwgchair@gmail.com>

Ready Issues

[2779^{\(i\)}](#). [networking.ts] Relax requirements on buffer sequence iterators

Section: 99 [networking.ts::buffer.reqmts.mutablebuffersequence] **Status:** [Ready](#) **Submitter:** Vinnie Falco **Opened:** 2016-10-05 **Last modified:** 2017-08-01

Priority: Not Prioritized

View all issues with [Ready](#) status.

Discussion:

Addresses: `networking.ts`

We propose to relax the **ForwardIterator** requirements of buffer sequences in [networking.ts] by allowing buffer sequence iterators to return rvalues when dereferenced, and skip providing operator->. A paraphrased explanation of why the referential equality rules of **ForwardIterator** are harmful to the buffer sequence requirements comes from [N4128](#), 3.3.7 "Ranges For The Standard Library":

The [networking.ts] dependence on **ForwardIterator** in the buffer sequence requirements ties together the traversal and access properties of iterators. For instance, no forward iterator may return an *rvalue* proxy when it is dereferenced; the **ForwardIterator** concept requires that unary operator* return an lvalue. This problem has serious consequences for lazy evaluation that applies transformations to buffer sequence elements on the fly. If the transformation function does not return an *lvalue*, the range's iterator can model no concept stronger than **InputIterator**, even if the resulting iterator could in theory support **BidirectionalIterator**. The result in practice is that most range adaptors today will not be compatible with [networking.ts], thereby limiting the types that [networking.ts] can be passed, for no good reason.

Consider a user defined function trim which lazily adapts a ConstBufferSequence, such that when iterating the buffers in the new sequence, each buffer appears one byte shorter than in the underlying sequence:

```
#include <boost/range/adaptor/transformed.hpp>

struct trim
{
    using result_type = const_buffer;
    result_type operator()(const_buffer b)
    {
        return const_buffer{b.data(), b.size() - 1};
    }
};

template <ConstBufferSequence>
auto
trim(ConstBufferSequence const& buffers)
{
    using namespace boost::adaptors;
    return buffers | transformed(trim{});
}
```

trim returns a **BidirectionalRange**, whose const_iterator returns an rvalue when dereferenced. This breaks the requirements of **ForwardIterator**. A solution that meets the referential equality rules of **ForwardIterator**, would be to evaluate the transformed sequence upon construction (for example, by storing each transformed const_buffer in a vector). Unfortunately this work-around is more expensive since it would add heap allocation which the original example avoids.

The requirement of **InputIterator** operator-> is also unnecessary for buffer sequence iterators, and should be removed. Because [networking.ts] only requires that a buffer sequence iterator's value_type be convertible to const_buffer or mutable_buffer, implementations of [networking.ts] cannot assume the existence of any particular member functions or data members other than an implicit conversion to const_buffer or mutable_buffer. Removing the requirement for operator-> to be present, provides additional relief from the referential equality requirements of **ForwardIterator** and allows transformations of buffer sequences to meet the requirements of buffer sequences.

This proposal imposes no changes on existing implementations of [networking.ts]. It does not change anything in the standard. The proposal is precise, minimal, and allows range adapters to transform buffer sequences in optimized, compatible ways.

[Issues processing Telecon 2016-10-07]

Status set to LEWG

[2017-02-21, Jonathan comments]

The use of the term "strict aliasing" in the issue discussion is misleading as that refers to type-based alias analysis in compilers, but the rule for ForwardIterators is related to referential equality and not strict aliasing.

[2017-02-22, Vinnie Falco comments]

We have eliminated the use of the term "strict aliasing" from the discussion.

[2017-07-10, Toronto, LEWG comments]

Status change: LEWG → Open.

Forward to LWG with the note that they may want to use "input +" instead of "bidirectional -". Unanimous yes.

[2017-07 Toronto Wednesday night issue processing]

Status to Ready

Proposed resolution:

This wording is relative to [N4588](#).

1. Modify 16.2.1 [buffer.reqmts.mutablebuffersequence] as indicated:

~~An iterator type meeting the requirements for bidirectional iterators (C++Std [bidirectional.iterators]) whose value type is convertible to mutable_buffer~~

An iterator type whose reference type is convertible to mutable buffer, and which satisfies all the requirements for bidirectional iterators (C++Std [bidirectional.iterators]) except that:

- a. there is no requirement that operator-> is provided, and
- b. there is no requirement that reference be a reference type.

2. Modify 16.2.2 [buffer.reqmts.constbuffersequence] as indicated:

~~An iterator type meeting the requirements for bidirectional iterators (C++Std [bidirectional.iterators]) whose value type is convertible to const_buffer;~~

An iterator type whose reference type is convertible to const buffer, and which satisfies all the requirements for bidirectional iterators (C++Std [bidirectional.iterators]) except that:

- a. there is no requirement that operator-> is provided, and
- b. there is no requirement that reference be a reference type.

[2870^{\(i\)}](#). Default value of parameter theta of polar should be dependent

Section: 29.5.7 [\[complex.value.ops\]](#) Status: [Ready](#) Submitter: Japan Opened: 2017-02-03 Last modified: 2017-08-01

Priority: Not Prioritized

View all other [issues](#) in [complex.value.ops].

View all issues with [Ready](#) status.

Discussion:

Addresses JP 25

Parameter theta of polar has the type of the template parameter. Therefore, it needs to change the default initial value to $\tau()$. The change of the declaration of this function in 29.5.1 [\[complex.syn\]](#) is accompanied by this change.

Proposed change:

```
template<class T> complex<T> polar(const T& rho, const T& theta = 0I());
```

[2017-02 pre-Kona]

(twice)

[2017-06-27 Moved to Tentatively Ready after 7 positive votes on c++std-lib.]

Proposed resolution:

This wording is relative to [N4659](#).

1. Modify 29.5.1 [\[complex.syn\]](#), header <complex> synopsis, as indicated:

```
template<class T> complex<T> polar(const T&, const T& = 0I());
```

2. Modify 29.5.7 [\[complex.value.ops\]](#) as indicated:

```
template<class T> complex<T> polar(const T& rho, const T& theta = 0I());
```

[2935^{\(i\)}](#). What should create_directories do when p already exists but is not a directory?

Section: 30.10.14.6 [\[fs.op.create_directories\]](#), 30.10.14.7 [\[fs.op.create_directory\]](#) **Status:** [Ready](#) **Submitter:** Billy Robert O'Neal III **Opened:** 2017-02-15 **Last modified:** 2017-07-15

Priority: 0

View all issues with [Ready](#) status.

Discussion:

The create_directory and create_directories functions have a postcondition that says is_directory(p), but it is unclear how they are supposed to provide this. Both of their effects say that they create a directory and return whether it was actually created. It is possible to interpret this as "if creation fails due to the path already existing, issue another system call to see if the path is a directory, and change the result if so" — but it seems unfortunate to require both Windows and POSIX to issue more system calls in this case.

In email discussion Davis Herring and Billy O'Neal discussed this issue and agreed that this was probably unintentional. Special thanks for Jonathan Wakely's suggested change to create_directories' Returns clause.

[2017-07 Toronto Thurs Issue Prioritization]

Priority 0; move to Ready

Proposed resolution:

This wording is relative to [N4640](#).

1. Make the following edits to 30.10.14.6 [\[fs.op.create_directories\]](#):

```
bool create_directories(const path& p);  
bool create_directories(const path& p, error_code& ec) noexcept;
```

-1- *Effects:* ~~Establishes the postcondition by calling~~ [Calls](#) create_directory() for ~~any~~[each](#) element of p that does not exist.

~~-2- Postconditions: is_directory(p).~~

-3- *Returns:* true if a new directory was created [for the directory p resolves to](#), otherwise false. The signature with argument ec returns false if an error occurs.

-4- *Throws:* As specified in 30.10.6 [\[fs.err.report\]](#).

-5- *Complexity:* $\mathcal{O}(n)$ where n is the number of elements of p ~~that do not exist~~.

2. Make the following edits to 30.10.14.7 [\[fs.op.create_directory\]](#):

```
bool create_directory(const path& p);  
bool create_directory(const path& p, error_code& ec) noexcept;
```

-1- *Effects:* ~~Establishes the postcondition by attempting to create~~ [Creates](#) the directory p resolves to, as if by POSIX mkdir() with a second argument of static_cast<int>(perms::all). Creation failure because p ~~resolves to an existing directory shall not be treated as~~[already exists is not](#) an error.

~~-2- Postconditions: is_directory(p)-~~

-3- Returns: true if a new directory was created, otherwise false. The signature with argument ec returns false if an error occurs.

-4- Throws: As specified in 30.10.6 [\[fs.err.report\]](#).

[2941^{\(i\)}](#). §[thread.req.timing] wording should apply to both member and namespace-level functions

Section: 33.2.4 [\[thread.req.timing\]](#) **Status:** [Ready](#) **Submitter:** Jonathan Mcdougall **Opened:** 2017-03-07 **Last modified:** 2017-07-15

Priority: 0

View all other [issues](#) in [thread.req.timing].

View all issues with [Ready](#) status.

Discussion:

In 33.2.4 [\[thread.req.timing\]](#), both /3 and /4 talk about "member functions whose names end in `_for`" and "`_until`", but these clauses also apply to `this_thread::sleep_for()` and `this_thread::sleep_until()`, which are namespace-level functions (30.3.2).

[2017-07 Toronto Wed Issue Prioritization]

Priority 0; Move to Ready

Proposed resolution:

This wording is relative to [N4640](#).

1. Modify 33.2.4 [\[thread.req.timing\]](#) as indicated::

[...]

-3- The ~~member~~ functions whose names end in `_for` take an argument that specifies a duration. [...]

-4- The ~~member~~ functions whose names end in `_until` take an argument that specifies a time point. [...]

[2944^{\(i\)}](#). LWG 2905 accidentally removed requirement that construction of the deleter doesn't throw an exception

Section: 23.11.1.2.1 [\[unique.ptr.single.ctor\]](#) **Status:** [Ready](#) **Submitter:** Tim Song **Opened:** 2017-03-11 **Last modified:** 2017-07-15

Priority: 0

View all other [issues](#) in [unique.ptr.single.ctor].

View all issues with [Ready](#) status.

Discussion:

The wording simplification in LWG [2905](#) accidentally deleted the requirement that construction of the deleter doesn't throw an exception. While this isn't the end of the world since any exception will just run into the `noexcept` on the constructor and result in a call to `std::terminate()`, the other `unique_ptr` constructors still have a similar no-exception *Requires:* clause, leaving us in the odd situation where throwing an exception results in undefined behavior for some constructors and `terminate()` for others. If guaranteeing `std::terminate()` on exception is desirable, that should be done across the board.

The proposed wording below simply restores the `nothrow` requirement along with the `Copy/MoveConstructible` requirement. Wording for the other alternative (guaranteed `std::terminate()`) can be produced if desired.

[2017-03-16, Daniel comments]

The publication of the new working draft is awaited, before proposed wording against that new working draft is formally possible.

[2017-05-03, Tim comments]

The suggested wording has been moved to the PR section now that the new working draft is available.

[2017-07 Toronto Wed Issue Prioritization]

Priority 0; Move to Ready

Proposed resolution:

This wording is relative to [N4659](#).

1. Insert a paragraph after 23.11.1.2.1 [\[unique.ptr.single.ctor\]](#) p11:

```
unique_ptr(pointer p, see below d1) noexcept;  
unique_ptr(pointer p, see below d2) noexcept;
```

-9- The signature of these constructors depends upon whether D is a reference type. If D is a non-reference type A, then the signatures are:

[...]

-10- If D is an lvalue reference type A&, then the signatures are:

[...]

-11- If D is an lvalue reference type const A&, then the signatures are:

[...]

??- Requires: For the first constructor, if D is not a reference type, D shall satisfy the requirements of CopyConstructible and such construction shall not exit via an exception. For the second constructor, if D is not a reference type, D shall satisfy the requirements of MoveConstructible and such construction shall not exit via an exception.

[2945^{\(i\)}](#). Order of template parameters in optional comparisons

Section: 23.6.8 [\[optional.comp.with.t\]](#) **Status:** [Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2017-03-13 **Last modified:** 2017-08-01

Priority: 2

View all other [issues](#) in [\[optional.comp.with.t\]](#).

View all issues with [Ready](#) status.

Discussion:

LWG [2934](#) added an additional template parameter to the comparison operators for `std::optional`, but the ones that compare U with `optional<T>` have the parameters backwards compared to the function parameters:

```
template <class T, class U>  
constexpr bool operator==(const U&, const optional<T>&);
```

Ville confirmed there's no particular reason for this, it's just how he wrote the proposed resolution, but as this has normative effect we should consider if we really want the template parameters and function parameters to be in different orders or not.

[2017-07-13, Casey Carter provides wording]

[2016-07, Toronto Thursday night issues processing]

Status to Ready

Proposed resolution:

This wording is relative to [N4659](#).

1. Modify 23.6.2 [\[optional.syn\]](#), `<optional>` synopsis, as indicated:

```
// 23.6.2 \[optional.comp.with.t\], comparison with T  
template <class T, class U> constexpr bool operator==(const optional<T>&, const U&);  
template <class T, class U> constexpr bool operator==(const U&, const optional<T>&);  
template <class T, class U> constexpr bool operator!=(const optional<T>&, const U&);  
template <class T, class U> constexpr bool operator!=(const U&, const optional<T>&);  
template <class T, class U> constexpr bool operator<(const optional<T>&, const U&);  
template <class T, class U> constexpr bool operator<(const U&, const optional<T>&);  
template <class T, class U> constexpr bool operator<=(const optional<T>&, const U&);  
template <class T, class U> constexpr bool operator<=(const U&, const optional<T>&);  
template <class T, class U> constexpr bool operator>(const optional<T>&, const U&);  
template <class T, class U> constexpr bool operator>(const U&, const optional<T>&);  
template <class T, class U> constexpr bool operator>=(const optional<T>&, const U&);  
template <class T, class U> constexpr bool operator>=(const U&, const optional<T>&);
```

2. Modify 23.6.8 [\[optional.comp.with.t\]](#) as indicated:

```
template <class T, class U> constexpr bool operator==(const U& v, const optional<U>& x);
-3- [...]

[...]

template <class T, class U> constexpr bool operator!=(const U& v, const optional<U>& x);
-7- [...]

[...]

template <class T, class U> constexpr bool operator<(const U& v, const optional<U>& x);
-11- [...]

[...]

template <class T, class U> constexpr bool operator<=(const U& v, const optional<U>& x);
-15- [...]

[...]

template <class T, class U> constexpr bool operator>(const U& v, const optional<U>& x);
-19- [...]

[...]

template <class T, class U> constexpr bool operator>=(const U& v, const optional<U>& x);
-23- [...]

[...]
```

[2948\(i\)](#). `unique_ptr` does not define `operator<<` for stream output

Section: 23.11.1 [\[unique.ptr\]](#) **Status:** [Ready](#) **Submitter:** Peter Dimov **Opened:** 2017-03-19 **Last modified:** 2017-07-15

Priority: 0

View all other [issues](#) in `[unique.ptr]`.

View all issues with [Ready](#) status.

Discussion:

`shared_ptr` does define `operator<<`, and `unique_ptr` should too, for consistency and usability reasons.

[2017-07 Toronto Wed Issue Prioritization]

Priority 0; move to Ready

Proposed resolution:

This wording is relative to [N4659](#).

1. Change 23.10.2 [\[memory.syn\]](#), header `<memory>` synopsis, as indicated:

```
namespace std {
[...]

// 23.11.1 \[unique.ptr\], class template unique_ptr
[...]
template <class T, class D>
bool operator>=(nullptr_t, const unique_ptr<T, D>& y);

template<class E, class T, class Y, class D>
basic_ostream<E, T>& operator<< (basic_ostream<E, T>& os, const unique_ptr<Y, D>& p);
[...]
}
```

2. Change 23.11.1 [\[unique_ptr\]](#), class template `unique_ptr` synopsis, as indicated:

```
namespace std {  
    [...]   
    template <class T, class D>  
    bool operator==(nullptr_t, const unique_ptr<T, D>& y);  
  
    template<class E, class T, class Y, class D>  
    basic_ostream<E, T>& operator<< (basic_ostream<E, T>& os, const unique_ptr<Y, D>& p);  
}
```

3. Add a new subclause following subclause 23.11.1.5 [\[unique_ptr.special\]](#) as indicated:

```
23.11.1.?? unique_ptr I/O [unique_ptr.io]  
  
template<class E, class T, class Y, class D>  
basic_ostream<E, T>& operator<< (basic_ostream<E, T>& os, const unique_ptr<Y, D>& p);  
  
-?- Effects: Equivalent to os << p.get().;  
  
-?- Returns: os.  
  
-?- Remarks: This function shall not participate in overload resolution unless os << p.get() is a valid  
expression.
```

[2950^{\(i\)}](#). `std::byte` operations are misspecified

Section: 21.2.5 [\[support.types.byteops\]](#) **Status:** [Ready](#) **Submitter:** Thomas Köppe **Opened:** 2017-03-24 **Last modified:** 2017-08-01

Priority: 1

View all issues with [Ready](#) status.

Discussion:

The operations for `std::byte` (21.2.5 [\[support.types.byteops\]](#)) are currently specified to have undefined behaviour in general cases, since the type of the expression `expr` that appears in `return byte(expr)` is obtained by the arithmetic conversion rules and has higher conversion rank than unsigned `char`. Therefore, the value of the expression may be outside the range of the enum (for example, consider `~0`), and by 8.2.9 [\[expr.static.cast\]](#) p10 the conversion results in undefined behaviour.

I believe the original intent of the specification could be expressed correctly with the following, more verbose sequence of casts. I will only give one representative example:

```
byte operator<<(byte b, IntType shift)
```

```
    Equivalent to: return byte(static_cast<unsigned char>(static_cast<unsigned char>(b) << shift));
```

[2017-06-27 P1 after 5 positive votes on c++std-lib.]

[2017-06-28, STL comments and provides wording]

This proposed resolution performs its work in unsigned `int`, which is immune to promotion. For `op=`, I'm avoiding unnecessary verbosity.

It stylistically uses `static_casts` instead of functional-style casts. All of the `static_casts` are intentional, although not all of them are strictly necessary. I felt that it was simpler to always follow the same pattern for type conversions, instead of skipping `static_casts` by taking advantage of the possible ranges of values. (I could prepare an alternative PR to avoid unnecessary casts.) I'm not `static_casting` the shift arguments, because of how 8.8 [\[expr.shift\]](#) works.

For `to_integer()`, there's a tiny question. According to 8.2.9 [\[expr.static.cast\]](#)/9, `static_casting` from `[128, 255]` bytes to signed (behavior) chars triggers unspecified behavior, whereas using unsigned `char` as an intermediate type would produce implementation-defined behavior, 7.8 [\[conv.integral\]](#)/3. This question is basically theoretical, and it's unaffected by this proposed resolution (which is just changing a functional-style cast to a `static_cast`).

[2016-07, Toronto Thursday night issues processing]

Status to Ready

Proposed resolution:

This wording is relative to [N4659](#).

1. Edit 21.2.5 [\[support.types.byteops\]](#) as indicated:

```
template <class IntType>
constexpr byte& operator<=(byte& b, IntType shift) noexcept;
```

-1- *Remarks:* This function shall not participate in overload resolution unless `is_integral_v<IntType>` is true.

-2- *Effects:* Equivalent to: return `b = b << shiftbyte(static_cast<unsigned char>(b) << shift);`

```
template <class IntType>
constexpr byte operator<<(byte b, IntType shift) noexcept;
```

-3- *Remarks:* This function shall not participate in overload resolution unless `is_integral_v<IntType>` is true.

-4- *Effects:* Equivalent to: return `static_cast<byte>(static_cast<unsigned char>(static_cast<unsigned int>(b) << shift)).byte(static_cast<unsigned char>(b) << shift);`

```
template <class IntType>
constexpr byte& operator>=(byte& b, IntType shift) noexcept;
```

-5- *Remarks:* This function shall not participate in overload resolution unless `is_integral_v<IntType>` is true.

-6- *Effects:* Equivalent to: return `b = b >> shiftbyte(static_cast<unsigned char>(b) >> shift);`

```
template <class IntType>
constexpr byte operator>>(byte b, IntType shift) noexcept;
```

-7- *Remarks:* This function shall not participate in overload resolution unless `is_integral_v<IntType>` is true.

-8- *Effects:* Equivalent to: return `static_cast<byte>(static_cast<unsigned char>(static_cast<unsigned int>(b) >> shift)).byte(static_cast<unsigned char>(b) >> shift);`

```
constexpr byte& operator|=(byte& l, byte r) noexcept;
```

-9- *Effects:* Equivalent to:

return `l = l | rbyte(static_cast<unsigned char>(l) | static_cast<unsigned char>(r));`

```
constexpr byte operator|(byte l, byte r) noexcept;
```

-10- *Effects:* Equivalent to:

return `static_cast<byte>(static_cast<unsigned char>(static_cast<unsigned int>(l) | static_cast<unsigned int>(r))).byte(static_cast<unsigned char>(l) | static_cast<unsigned char>(r));`

```
constexpr byte& operator&=(byte& l, byte r) noexcept;
```

-11- *Effects:* Equivalent to:

return `l = l & rbyte(static_cast<unsigned char>(l) & static_cast<unsigned char>(r));`

```
constexpr byte operator&(byte l, byte r) noexcept;
```

-12- *Effects:* Equivalent to:

return `static_cast<byte>(static_cast<unsigned char>(static_cast<unsigned int>(l) & static_cast<unsigned int>(r))).byte(static_cast<unsigned char>(l) & static_cast<unsigned char>(r));`

```
constexpr byte& operator^=(byte& l, byte r) noexcept;
```

-13- *Effects:* Equivalent to:

return `l = l ^ rbyte(static_cast<unsigned char>(l) ^ static_cast<unsigned char>(r));`

```
constexpr byte operator^(byte l, byte r) noexcept;
```

-14- *Effects:* Equivalent to:

return `static_cast<byte>(static_cast<unsigned char>(static_cast<unsigned int>(l) ^ static_cast<unsigned int>(r))).byte(static_cast<unsigned char>(l) ^ static_cast<unsigned char>(r));`

```
constexpr byte operator~(byte b) noexcept;
```

-15- *Effects:* Equivalent to: return `static_cast<byte>(static_cast<unsigned char>(static_cast<unsigned int>(b) ^ 0xFF)).byte(~static_cast<unsigned char>(b));`


```
template <class IntType>
constexpr IntType to_integer(byte b) noexcept;
```

-16- *Remarks:* This function shall not participate in overload resolution unless `is_integral_v<IntType>` is true.

-17- *Effects:* Equivalent to: return `static_cast<IntType>IntType(b);`

2952⁽ⁱ⁾. `iterator_traits` should work for pointers to cv T

Section: 27.4.1 [\[iterator.traits\]](#) **Status:** [Ready](#) **Submitter:** Billy Robert O'Neal III **Opened:** 2017-03-27 **Last modified:** 2017-07-15

Priority: 0

View other [active issues](#) in `[iterator.traits]`.

View all other [issues](#) in `[iterator.traits]`.

View all issues with [Ready](#) status.

Discussion:

`iterator_traits` accepts pointer to volatile T*, but then says that the `value_type` is volatile T, instead of T, which is inconsistent for what it does for pointer to const T. We should either reject volatile outright or give the right answer.

[2017-03-30, David Krauss comments]

volatile pointers may not be well-behaved random-access iterators. When simple access incurs side effects, the multiple-pass guarantee depends on underlying (hardware) semantics.

[2017-07 Toronto Wed Issue Prioritization]

Priority 0; move to Ready

Proposed resolution:

This wording is relative to [N4659](#).

1. Change 27.3 [\[iterator.synopsis\]](#) as indicated:

```
// 27.4 \[iterator.primitives\], primitives
template<class Iterator> struct iterator_traits;
template<class T> struct iterator_traits<T*>;
template<class T> struct iterator_traits<const T*>;
```

2. Change 27.4.1 [\[iterator.traits\]](#) as indicated:

-3- It is specialized for pointers as

```
namespace std {
    template<class T> struct iterator_traits<T*> {
        using difference_type = ptrdiff_t;
        using value_type = remove\_cv\_t<T>;
        using pointer = T*;
        using reference = T&;
        using iterator_category = random_access_iterator_tag;
    };
}
```

~~and for pointers to const as~~

```
namespace std {
    template<class T> struct iterator_traits<const T*> {
        using difference_type = ptrdiff_t;
        using value_type = T;
        using pointer = const T*;
        using reference = const T&;
        using iterator_category = random_access_iterator_tag;
    };
}
```

2953⁽ⁱ⁾. LWG 2853 should apply to `deque::erase` too

Section: 26.3.8.4 [\[deque.modifiers\]](#) **Status:** [Ready](#) **Submitter:** Tim Song **Opened:** 2017-03-30 **Last modified:** 2017-07-15

Priority: 0

View all other [issues](#) in [\[deque.modifiers\]](#).

View all issues with [Ready](#) status.

Discussion:

Most of the discussion of LWG [2853](#) applies, mutatis mutandis, to `deque::erase`. The relevant requirements table requires neither `Copy/MoveInsertable` nor `Copy/MoveConstructible` for the erase operations, so there's no way a copy/move constructor can safely be called.

And "assignment operator or move assignment operator" is just "assignment operator", since "move assignment operator" is just a species of "assignment operator".

[2017-07 Toronto Wed Issue Prioritization]

Priority 0; Move to Ready

Proposed resolution:

This wording is relative to [N4659](#).

1. Change 26.3.8.4 [\[deque.modifiers\]](#) as indicated:

```
iterator erase(const_iterator position);
iterator erase(const_iterator first, const_iterator last);
void pop_front();
void pop_back();
```

-4- *Effects:* [...]

-5- *Complexity:* The number of calls to the destructor of T is the same as the number of elements erased, but the number of calls to the assignment operator of T is no more than the lesser of the number of elements before the erased elements and the number of elements after the erased elements.

-6- *Throws:* Nothing unless an exception is thrown by the ~~copy constructor, move constructor,~~ assignment operator, ~~or move assignment operator~~ of T .

[2964^{\(i\)}](#). Apparently redundant requirement for `dynamic_pointer_cast`

Section: 23.11.2.2.9 [\[util.smartptr.shared.cast\]](#) **Status:** [Ready](#) **Submitter:** Tim Song **Opened:** 2017-05-11 **Last modified:** 2017-07-11

Priority: 0

View all other [issues](#) in [\[util.smartptr.shared.cast\]](#).

View all issues with [Ready](#) status.

Discussion:

Currently 23.11.2.2.9 [\[util.smartptr.shared.cast\]](#)/4 says:

Requires: The expression `dynamic_cast<T*>((U*)nullptr)` shall be well formed and shall have well defined behavior.

A `dynamic_cast` of a null pointer, if well-formed, always has well-defined behavior: it returns a null pointer. The second part is therefore redundant as currently worded. The C++14 version, on the other hand, requires `dynamic_cast<T*>(r.get())` to have well-defined behavior, which actually adds something: it requires the user to not trigger the undefined case in [\[class.ctor\]/5](#), for instance.

[2017-07 Toronto Monday issue prioritization]

Priority 0; move to Ready

Proposed resolution:

This wording is relative to [N4659](#).

1. Edit 23.11.2.2.9 [\[util.smartptr.shared.cast\]](#) as indicated:

```
shared_ptr<T> dynamic_pointer_cast(const shared_ptr<U>& r) noexcept;
```

-4- *Requires:* The expression `dynamic_cast<T*>((U*)0)` shall be well formed. The expression `dynamic_cast<typename shared_ptr<T>::element_type*>(r.get())` shall be well formed and shall have well defined behavior.

[...]

[2965\(i\)](#). **Non-existing path::native_string() in filesystem_error::what() specification**

Section: 99 [filesystem_error.members] **Status:** [Ready](#) **Submitter:** Daniel Krüger **Opened:** 2017-05-22 **Last modified:** 2017-07-11

Priority: 0

View all issues with [Ready](#) status.

Discussion:

As pointed out by Jonathan Wakely and Bo Persson, 99 [filesystem_error.members]/7 refers to a non-existing function `path::native_string`:

Returns: A string containing `runtime_error::what()`. The exact format is unspecified. Implementations are encouraged but not required to include `path1.native_string()` if not empty, `path2.native_string()` if not empty, and `system_error::what()` strings in the returned string.

Existing implementations differ, as Jonathan also determined:

- Boost.Filesystem uses `path::string()`.
- Libstdc++ uses `path::string()`.
- MSVC++/Dinkumware uses `path::u8string()`.
- It seems that libc++ doesn't include the paths in `what()`.

We've had `native_string()` in the spec since [N3239](#) (where it already didn't match any existing path function at that time).

Before that it was `file_string()` in [N1975](#) (within that specification path was a template that was parametrized in the character type).

Since it can't be `path::native()` because that might be the wrong type, one of `path::string()` or `path::u8string()` seems appropriate.

Albeit the wording is just a non-binding encouragement to implementations, the decision on this matter should not be considered editorially due to the existing implementation variance. Any official resolution of the current state could cause a reconsideration of existing implementations, and therefore it should be documented.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4659](#).

1. Edit 99 [filesystem_error.members] as indicated:

```
const char* what() const noexcept override;
```

-7- *Returns:* A string containing `runtime_error::what()`. The exact format is unspecified. Implementations are encouraged but not required to include `path1.native.string()` if not empty, `path2.native.string()` if not empty, and `system_error::what()` strings in the returned string.

[2017-05-25, Jonathan comments and suggests an alternative resolution]

The revised wording changes leave it up to the implementation which of the native format observers to use. The "if not empty" seems redundant, because if the path is empty then there's nothing to include anyway, but the proposed resolution preserves it.

[2017-07 Toronto Monday issue prioritization]

Priority 0; move to Ready

Proposed resolution:

This wording is relative to [N4659](#).

1. Edit 99 [filesystem_error.members] as indicated:

```
const char* what() const noexcept override;
```

-7- Returns: A string containing `runtime_error::what()`. The exact format is unspecified. Implementations are encouraged but not required to include `path1.native_string()` if not empty, `path2.native_string()` if not empty, and `system_error::what()` strings the `system_error::what()` string and the pathnames of `path1` and `path2` in the native format in the returned string.

2972⁽ⁱ⁾. What is `is_trivially_destructible_v<int>`?

Section: 23.15.4.3 [\[meta.unary.prop\]](#) **Status:** [Ready](#) **Submitter:** Richard Smith **Opened:** 2017-06-01 **Last modified:** 2017-08-01

Priority: Not Prioritized

View other [active issues](#) in [\[meta.unary.prop\]](#).

View all other [issues](#) in [\[meta.unary.prop\]](#).

View all issues with [Ready](#) status.

Discussion:

The spec for `is_trivially_destructible` says the value is true if "`is_destructible_v<T>` is true and the indicated destructor is known to be trivial."

For a case like `is_trivially_destructible_v<int>`, there is no indicated destructor, so it's unclear what value the trait would have but the most plausible reading of these words is that it should be false. However, I'm confident the intent is that this trait should yield true in that situation, and that's what all the implementations I can find actually do.

[2017-06-14, Daniel and Jonathan provide wording]

[2017-07-05 Moved to Tentatively Ready after 5 positive votes on c++std-lib.]

Proposed resolution:

This wording is relative to [N4659](#).

1. Change 23.15.4.3 [\[meta.unary.prop\]](#), Table 42 — "Type property predicates", as indicated:

Table 22 — Type property predicates		
Template	Condition	Preconditions
...		
template <class T> struct is_trivially_destructible;	<code>is_destructible_v<T></code> is true and the indicated destructor is known to be trivial. remove all extents t<T> is either a non-class type or a class type with a trivial destructor.	T shall be a complete type, cv void, or an array of unknown bound.
...		

2976⁽ⁱ⁾. Dangling uses_allocator specialization for packaged_task

Section: 33.6.10 [\[futures.task\]](#), 33.6.2 [\[future.syn\]](#), 33.6.10.2 [\[futures.task.nonmembers\]](#) **Status:** [Ready](#) **Submitter:** Tim Song **Opened:** 2017-06-13 **Last modified:** 2017-08-01

Priority: 3

View all other [issues](#) in [\[futures.task\]](#).

View all issues with [Ready](#) status.

Discussion:

When LWG [2921](#) removed allocator support from `packaged_task`, it forgot to remove the `uses_allocator` partial specialization.

[2017-06-26 Moved to Tentatively Ready after 6 positive votes on c++std-lib.]

[2017-06-26, Billy O'Neal reopens]

I think [2921](#) was resolved in error. If `promise<T>` can have an allocator, there's no reason for `packaged_task<T>` to not have one. If we remove it from `packaged_task` we should remove it from `promise` as well.

Note that I am not objecting to removing allocator support here, I'm objecting to the "remove it because this looks like `std::function`" case. `packaged_task` has none of the `std::function` problems because the function inside a given `packaged_task` is not reassignable.

If LWG decides to remove allocator support here then there are more bits that need to be struck, e.g. [\[futures.task.members\]\(5.3\)](#).

[2017-06-26, Tim updates P/R to remove more dangling bits.]

The additional point in the P/R effectively reverts the second part of the resolution of [2752](#).

The alternative resolution for this issue is, of course, to just revert the resolution of 2921. In that case [2245](#) needs to be reopened.

[2016-07, Toronto Saturday afternoon issues processing]

Status to Ready

Proposed resolution:

This wording is relative to [N4659](#).

1. Modify 33.6.2 [\[future.syn\]](#), header `<future>` synopsis, and 33.6.10 [\[futures.task\]](#), class template `packaged_task` synopsis, as indicated:

```
template <class R, class Alloc>  
struct uses_allocator<packaged_task<R>, Alloc>;
```

2. Modify 33.6.10.2 [\[futures.task.nonmembers\]](#) as indicated:

```
template <class R, class Alloc>  
struct uses_allocator<packaged_task<R>, Alloc>  
: true_type { };
```

~~-2- Requires: Alloc shall be an Allocator (20.5.3.5).~~

3. Modify 33.6.10.1 [\[futures.task.members\]](#)/5 as indicated:

```
template <class F>  
packaged_task(F&& F);
```

-2- Requires: [...]

-3- Remarks: [...]

-4- Effects: [...]

-5- Throws:

~~Any exceptions thrown by the copy or move constructor of f, or~~

~~For the first version, bad_alloc if memory for the internal data structures could not be allocated.~~

~~For the second version, any exceptions thrown by allocator_traits<Allocator>::template
rebind_traits<unspecified>::allocate.~~

[2977^{\(i\)}](#). `unordered_meow::merge()` has incorrect *Throws:* clause

Section: 26.2.7 [\[unord.req\]](#) Status: [Ready](#) Submitter: Tim Song Opened: 2017-06-14 Last modified: 2017-07-11

Priority: 0

View other [active issues](#) in [\[unord.req\]](#).

View all other [issues](#) in [\[unord.req\]](#).

View all issues with [Ready](#) status.

Discussion:

As pointed out in [this StackOverflow question](#), `unordered_{map,multimap,set,multiset}::merge()` may need to rehash to maintain its `max_load_factor` invariant, which may require allocation, which may throw.

[2017-07 Toronto Monday issue prioritization]

Priority 0; move to Ready

Proposed resolution:

This wording is relative to [N4659](#).

1. In 26.2.7 [\[unord.req\]](#), edit Table 91 "Unordered associative container requirements" as indicated:

Table 91 — Unordered associative container requirements (in addition to container)			
Expression	Return type	Assertion/note pre-/post-condition	Complexity
...			
a.merge(a2)	void	<i>Requires:</i> a.get_allocator() == a2.get_allocator(). Attempts to extract each element in a2 and insert it into a using the hash function and key equality predicate of a. In containers with unique keys, if there is an element in a with key equivalent to the key of an element from a2, then that element is not extracted from a2. <i>Postconditions:</i> Pointers and references to the transferred elements of a2 refer to those same elements but as members of a. Iterators referring to the transferred elements and all iterators referring to a will be invalidated, but iterators to elements remaining in a2 will remain valid. <i>Throws:</i> Nothing unless the hash function or key equality predicate throws.	Average case $\mathcal{O}(N)$, where N is a2.size(). Worst case $\mathcal{O}(N \cdot a.size() + N)$.
...			

[2978^{\(i\)}](#). Hash support for pmr::string and friends

Section: 24.3.5 [\[basic.string.hash\]](#), 24.3.1 [\[string.syn\]](#) **Status:** [Ready](#) **Submitter:** Tim Song **Opened:** 2017-06-14 **Last modified:** 2017-07-11

Priority: 0

View all issues with [Ready](#) status.

Discussion:

In most cases, std::pmr::meow is a drop-in replacement for std::meow. The exception is std::pmr::{w,u16,u32}string, because unlike their std:: counterparts, they don't come with enabled std::hash specializations.

The P/R below simply adds std::hash specializations for those four typedefs. An alternative approach, for which wording can be produced if desired, is to make the hash specializations for basic_string allocator-agnostic, similar to the partial specialization of hash for vector<bool>.

[2017-07 Toronto Monday issue prioritization]

Priority 0; move to Ready

Proposed resolution:

This wording is relative to [N4659](#).

1. Edit 24.3.1 [\[string.syn\]](#), header <string> synopsis, as indicated:

```
namespace pmr {
    template <class charT, class traits = char_traits<charT>>
        using basic_string = std::basic_string<charT, traits, polymorphic_allocator<charT>>;

    using string      = basic_string<char>;
    using u16string   = basic_string<char16_t>;
    using u32string   = basic_string<char32_t>;
    using wstring     = basic_string<wchar_t>;
}

// 24.3.5 \[basic.string.hash\], hash support
template<class T> struct hash;
template<> struct hash<string>;
template<> struct hash<u16string>;
template<> struct hash<u32string>;
template<> struct hash<wstring>;
template<> struct hash<pmr::string>;
template<> struct hash<pmr::u16string>;
template<> struct hash<pmr::u32string>;
template<> struct hash<pmr::wstring>;

namespace pmr {
    template <class charT, class traits = char_traits<charT>>
        using basic_string = std::basic_string<charT, traits, polymorphic_allocator<charT>>;
```

```

using string      = basic_string<char>;
using u16string   = basic_string<char16_t>;
using u32string   = basic_string<char32_t>;
using wstring     = basic_string<wchar_t>;
}

```

2. Edit 24.3.5 [\[basic.string.hash\]](#) as indicated:

```

template<> struct hash<string>;
template<> struct hash<u16string>;
template<> struct hash<u32string>;
template<> struct hash<wstring>;
template<> struct hash<pmr::string>;
template<> struct hash<pmr::u16string>;
template<> struct hash<pmr::u32string>;
template<> struct hash<pmr::wstring>;

```

-1- If S is one of these string types, SV is the corresponding string view type, and s is an object of type S, then
hash<S>()(s) == hash<SV>()(SV(s)).

2979(i). aligned_union should require complete object types

Section: 23.15.7.6 [\[meta.trans.other\]](#) **Status:** [Ready](#) **Submitter:** Tim Song **Opened:** 2017-06-14 **Last modified:** 2017-07-11

Priority: 0

View all other [issues](#) in [\[meta.trans.other\]](#).

View all issues with [Ready](#) status.

Discussion:

aligned_union's description doesn't, but should, require the types provided to be complete object types.

[2017-07 Toronto Monday issue prioritization]

Priority 0; move to Ready

Proposed resolution:

This wording is relative to [N4659](#).

1. In 23.15.7.6 [\[meta.trans.other\]](#), edit Table 50 "Other transformations" as indicated:

Table 50 — Other transformations

Template	Comments
	[...]
template <size_t Len, class... Types> struct aligned_union;	The member typedef type shall be a POD type suitable for use as uninitialized storage for any object whose type is listed in Types; its size shall be at least Len. The static member alignment_value shall be an integral constant of type size_t whose value is the strictest alignment of all types listed in Types. <i>Requires:</i> At least one type is provided. Each type in the parameter pack Types shall be a complete object type.
	[...]

2980(i). Cannot compare_exchange empty pointers

Section: 23.11.2.6 [\[util.smartptr.shared.atomic\]](#) **Status:** [Ready](#) **Submitter:** Alisdair Meredith **Opened:** 2017-06-15 **Last modified:** 2017-08-01

Priority: Not Prioritized

View all other [issues](#) in [\[util.smartptr.shared.atomic\]](#).

View all issues with [Ready](#) status.

Discussion:

[\[util.smartptr.shared.atomic\]](#) p35 states that two shared pointers are equivalent if they store the same pointer value, and share *ownership*. As empty shared pointers never share ownership, it is not possible to replace an empty shared pointer using the atomic compare_exchange API.

Note that through aliasing, empty shared pointers may still point to different objects, and any resolution must allow for that case too.

[2017-06-26 Moved to Tentatively Ready after 5 positive votes on c++std-lib.]

Proposed resolution:

This wording is relative to [N4659](#).

1. Edit 23.11.2.6 [\[util.smartptr.shared.atomic\]](#) as indicated:

```
template<class T>
    bool atomic_compare_exchange_weak_explicit(
        shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w,
        memory_order success, memory_order failure);
template<class T>
    bool atomic_compare_exchange_strong_explicit(
        shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w,
        memory_order success, memory_order failure);

[...]
```

-35- Remarks: Two shared_ptr objects are equivalent if they store the same pointer value and share ownership, or if they store the same pointer value and both are empty. The weak form may fail spuriously. See 32.6.1.

[2981](#)⁽ⁱ⁾. Remove redundant deduction guides from standard library

Section: 23.14.5 [\[refwrap\]](#), 33.4.4.1 [\[thread.lock.guard\]](#), 33.4.4.2 [\[thread.lock.scoped\]](#), 33.4.4.3 [\[thread.lock.unique\]](#), 33.4.4.4 [\[thread.lock.shared\]](#) **Status:** [Ready](#) **Submitter:** Mike Spertus **Opened:** 2017-06-15 **Last modified:** 2017-07-11

Priority: 0

View other [active issues](#) in [\[refwrap\]](#).

View all other [issues](#) in [\[refwrap\]](#).

View all issues with [Ready](#) status.

Discussion:

There are several deduction guides added to the standard library by [P0433R2](#) that have no effect probably because LWG had not considered late changes to core wording that automatically adds a "copy deduction candidate" (16.3.1.8 [\[over.match.class.deduct\]](#)) that renders these explicit guides moot.

[2017-07 Toronto Monday issue prioritization]

Priority 0; move to Ready

Proposed resolution:

This wording is relative to [N4659](#).

1. Edit 23.14.5 [\[refwrap\]](#), end of class template reference_wrapper synopsis, as indicated:

```
template<class T>
    reference_wrapper(reference_wrapper<T>) -> reference_wrapper<T>;
```

2. Edit 33.4.4.1 [\[thread.lock.guard\]](#), end of class template lock_guard synopsis, as indicated:

```
template<class Mutex> lock_guard(lock_guard<Mutex>) -> lock_guard<Mutex>;
```

3. Edit 33.4.4.2 [\[thread.lock.scoped\]](#), end of class template scoped_lock synopsis, as indicated:

```
template<class... MutexTypes>
    scoped_lock(scoped_lock<MutexTypes...>) -> scoped_lock<MutexTypes...>;
```

4. Edit 33.4.4.3 [\[thread.lock.unique\]](#), end of class template unique_lock synopsis, as indicated:

```
template<class Mutex> unique_lock(unique_lock<Mutex>) -> unique_lock<Mutex>;
```

5. Edit 33.4.4.4 [\[thread.lock.shared\]](#), end of class template shared_lock synopsis, as indicated:

```
template<class Mutex> shared_lock(shared_lock<Mutex>) -> shared_lock<Mutex>;
```

[2982^{\(i\)}](#). Making size_type consistent in associative container deduction guides

Section: 26.5.6.1 [\[unord.set.overview\]](#), 26.5.7.1 [\[unord.multiset.overview\]](#) **Status:** [Ready](#) **Submitter:** Mike Spertus **Opened:** 2017-06-16 **Last modified:** 2017-08-01

Priority: 2

View all issues with [Ready](#) status.

Discussion:

Due to an incompletely implemented change in Kona, some of the size_type deduction guides say something like:

A size_type parameter type in an unordered_set deduction guide refers to the size_type member type of the primary unordered_set template

while others say

A size_type parameter type in an unordered_map deduction guide refers to the size_type member type of the type deduced by the deduction guide.

Clearly they should both be the same. My recollection is that the intent of the committee was to change them all to be the latter. Note, however, that this issue may be mooted if the suggestions in the upcoming [P0433R3](#) paper are adopted as a DR in Toronto.

[2017-07 Toronto Monday issue prioritization]

Priority 2; Mike is preparing an updated paper — currently named [P0433R3](#).

[2016-07, Toronto Saturday afternoon issues processing]

Status to Ready; Marshall to check with Mike about his paper

Proposed resolution:

This wording is relative to [N4659](#).

1. Edit 26.5.6.1 [\[unord.set.overview\]](#) as indicated:

-4- A size_type parameter type in an unordered_set deduction guide refers to the size_type member type of the ~~primary unordered_set template~~ type deduced by the deduction guide.

2. Edit 26.5.7.1 [\[unord.multiset.overview\]](#) as indicated:

-4- A size_type parameter type in an unordered_multiset deduction guide refers to the size_type member type of the ~~primary unordered_multiset template~~ type deduced by the deduction guide.

[2988^{\(i\)}](#). Clause 32 cleanup missed one typename

Section: 32.2 [\[atomics.syn\]](#) **Status:** [Ready](#) **Submitter:** Jens Maurer **Opened:** 2017-06-25 **Last modified:** 2017-08-01

Priority: 0

View other [active issues](#) in [atomics.syn].

View all other [issues](#) in [atomics.syn].

View all issues with [Ready](#) status.

Discussion:

[P0558R1](#) missed updating one of the std::atomic_exchange signatures to avoid independent deduction for T on the second parameter.

[2017-06-26 Moved to Tentatively Ready after 6 positive votes on c++std-lib.]

Proposed resolution:

This wording is relative to [N4659](#).

1. Edit 32.2 [\[atomics.syn\]](#), header <atomic> synopsis, as indicated:

```
template<class T>
T atomic_exchange(volatile atomic<T>*, typename atomic<T>::value_type) noexcept;
```

[2993\(i\)](#). `reference_wrapper<T>` conversion from `T&&`

Section: 23.14.5 [\[refwrap\]](#) **Status:** [Ready](#) **Submitter:** Tim Song **Opened:** 2017-06-28 **Last modified:** 2017-08-01

Priority: 3

View other [active issues](#) in [\[refwrap\]](#).

View all other [issues](#) in [\[refwrap\]](#).

View all issues with [Ready](#) status.

Discussion:

`reference_wrapper<T>` has a deleted constructor taking `T&&` in order to prevent accidentally wrapping an rvalue (which can otherwise happen with the `reference_wrapper(T&)` constructor if `T` is a non-volatile const-qualified type). Unfortunately, a deleted constructor can still be used to form implicit conversion sequences, so the deleted `T&&` constructor has the (presumably unintended) effect of creating an implicit conversion sequence from a `T` rvalue to a `reference_wrapper<T>`, even though such a conversion would be ill-formed if actually used. This is visible in overload resolution:

```
void meow(std::reference_wrapper<int>); // #1
void meow(convertible_from_int); // #2
meow(0); // error, ambiguous; would unambiguously call #2 if #1 instead took int&
```

and in conditional expressions (and hence `std::common_type`) after [core issue 1895](#):

```
std::reference_wrapper<int> purr();

auto x = true? purr() : 0; // error, ambiguous: ICS exists from int prvalue to
                          // reference_wrapper<int> and from reference_wrapper<int> to int

using t = std::common_type_t<std::reference_wrapper<int>, int>; // error: no member 'type' because the conditional
                                                                // expression is ill-formed
```

The latter in turn interferes with the use of `reference_wrapper` as a proxy reference type with proxy iterators.

We should ensure that there is no implicit conversion sequence from `T` rvalues to `reference_wrapper<T>`, not just that the conversion will be ill-formed when used. This can be done by using a suitably constrained constructor template taking a forwarding reference instead of the current pair of constructors taking `T&` and `T&&`.

[2017-06-29, Tim adds P/R and comments]

The draft P/R below uses a conditional `noexcept` specification to ensure that converting a `T&` to a `reference_wrapper<T>` remains `noexcept` and make it not usable when the source type is a `reference_wrapper` of the same type so as to avoid affecting `is_trivially_constructible`. It adds a deduction guide as the new constructor template will not support class template argument deduction.

The constructor template has the additional effect of making `reference_wrapper<T>` convertible from everything that is convertible to `T&`. This implies, for instance, that `reference_wrapper<int>` is now convertible to `reference_wrapper<const int>` when it wasn't before (the conversion would have required two user-defined conversions previously). This more closely emulates the behavior of an actual reference, but does represent a change to the existing behavior.

If perfectly emulating the existing behavior is desired, a conditionally-explicit constructor that is only implicit if `T` is reference-compatible with `remove_reference_t<U>` (see 11.6.3 [\[dcl.init.ref\]](#)) can be used.

[2017-07 Toronto Tuesday PM issue prioritization]

Priority 3; what else in the library does this affect? `ref` or `cref`?

[2016-07, Toronto Saturday afternoon issues processing]

Status to Ready.

Proposed resolution:

This wording is relative to [N4659](#).

1. Edit 23.14.5 [\[refwrap\]](#), class template `reference_wrapper` synopsis, as indicated:

```

namespace std {
    template <class T> class reference_wrapper {
    [...]
        // construct/copy/destroy
        reference_wrapper(T&) noexcept;
        reference_wrapper(T&&) = delete; // do not bind to temporary objects
        template <class U>
            reference_wrapper(U&&) noexcept(see below);
    [...]
    };
    template <class T>
        reference_wrapper(T&) -> reference_wrapper<T>;
    [...]
}

```

2. Edit 23.14.5.1 [refwrap.const]/1 as indicated:

```
reference_wrapper(T& t) noexcept;
```

~~-1- Effects: Constructs a reference wrapper object that stores a reference to t.~~

```

template<class U>
    reference_wrapper(U&& u) noexcept(see below);

```

~~-?- Remarks: Let FUN denote the exposition-only functions~~

```

void FUN(T&) noexcept;
void FUN(T&&) = delete;

```

~~This constructor shall not participate in overload resolution unless the expression FUN(declval<U>()) is well-formed and is same v<decay t<U>, reference_wrapper> is false. The expression inside noexcept is equivalent to noexcept(FUN(declval<U>())).~~

~~-?- Effects: Creates a variable r as if by T& r = std::forward<U>(u), then constructs a reference wrapper object that stores a reference to r.~~

2998⁽ⁱ⁾. Requirements on function objects passed to {forward_, }list-specific algorithms

Section: 26.3.10.5 [list.ops], 26.3.9.6 [forwardlist.ops] **Status:** Ready **Submitter:** Tim Song **Opened:** 2017-07-07 **Last modified:** 2017-07-11

Priority: 0

View other active issues in [list.ops].

View all other issues in [list.ops].

View all issues with Ready status.

Discussion:

Some specialized algorithms for forward_list and list take template parameters named Predicate, BinaryPredicate, or Compare. However, there's no wording importing the full requirements for template type parameters with such names from 28.3 [algorithms.requirements] and 28.7 [alg.sorting], which means, for instance, that there appears to be no rule prohibiting Compare from modifying its arguments, because we only refer to 28.7 [alg.sorting] for the definition of strict weak ordering. Is that intended?

[2017-07 Toronto Tuesday PM issue prioritization]

Priority 0; status to Ready

Proposed resolution:

This wording is relative to N4659.

1. Edit 26.3.10.5 [list.ops] as indicated:

~~-1- Since lists allow fast insertion and erasing from the middle of a list, certain operations are provided specifically for them.²⁵⁹⁾~~ In this subclause, arguments for a template parameter named Predicate or BinaryPredicate shall meet the corresponding requirements in 28.3 [algorithms.requirements]. For merge and sort, the definitions and requirements in 28.7 [alg.sorting] apply.

2. Edit 26.3.10.5 [list.ops] as indicated:

```
void merge(list& x);
void merge(list&& x);
template <class Compare> void merge(list& x, Compare comp);
template <class Compare> void merge(list&& x, Compare comp);
```

-22- Requires: ~~comp shall define a strict weak ordering (28.7 [alg.sorting]), and b~~Both the list and the argument list shall be sorted ~~according to this ordering~~with respect to the comparator operator< (for the first two overloads) or comp (for the last two overloads).

3. Delete 26.3.10.5 [list.ops]/28 as redundant:

```
void sort();
template <class Compare> void sort(Compare comp);
```

~~-28- Requires: operator< (for the first version) or comp (for the second version) shall define a strict weak ordering (28.7 [alg.sorting]).~~

4. Insert a new paragraph at the beginning of 26.3.9.6 [forwardlist.ops]:

~~-?- In this subclause, arguments for a template parameter named Predicate or BinaryPredicate shall meet the corresponding requirements in 28.3 [algorithms.requirements]. For merge and sort, the definitions and requirements in 28.7 [alg.sorting] apply.~~

```
void splice_after(const_iterator position, forward_list& x);
void splice_after(const_iterator position, forward_list&& x);
```

[...]

5. Edit 26.3.9.6 [forwardlist.ops] as indicated:

```
void merge(forward_list& x);
void merge(forward_list&& x);
template <class Compare> void merge(forward_list& x, Compare comp);
template <class Compare> void merge(forward_list&& x, Compare comp);
```

-22- Requires: ~~comp defines a strict weak ordering (28.7 [alg.sorting]), and~~ *this and x are both sorted ~~according to this ordering~~with respect to the comparator operator< (for the first two overloads) or comp (for the last two overloads). get_allocator() == x.get_allocator().

6. Delete 26.3.9.6 [forwardlist.ops]/23 as redundant:

```
void sort();
template <class Compare> void sort(Compare comp);
```

~~-23- Requires: operator< (for the version with no arguments) or comp (for the version with a comparison argument) defines a strict weak ordering (28.7 [alg.sorting]).~~

3001⁽ⁱ⁾. weak_ptr::element_type needs remove_extent_t

Section: 23.11.2.3 [util.smartptr.weak] **Status:** [Tentatively Ready](#) **Submitter:** Stephan T. Lavavej **Opened:** 2017-07-14 **Last modified:** 2017-08-01

Priority: 0

View all other [issues](#) in [util.smartptr.weak].

View all issues with [Tentatively Ready](#) status.

Discussion:

C++17's shared_ptr<T>::element_type is remove_extent_t<T>, but weak_ptr<T>::element_type is T. They should be the same, but this was lost over time.

First, [N4562](#) "Working Draft, C++ Extensions for Library Fundamentals, Version 2" 8.2.2 [memory.smartptr.weak] specified:

```
namespace std {
namespace experimental {
inline namespace fundamentals_v2 {

template<class T> class weak_ptr {
public:
    typedef typename remove_extent_t<T> element_type;
```

(The typename here was spurious.)

Then, [P0220R1](#) "Adopt Library Fundamentals V1 TS Components for C++17 (R1)" listed:

- 8.2.2 Class template `weak_ptr`
 - 8.2.2.1 `weak_ptr` constructors

This obscured the fact that the Library Fundamentals TS had altered `weak_ptr::element_type`.

Finally, [P0414R2](#) "Merging `shared_ptr` changes from Library Fundamentals to C++17" missed the change to `weak_ptr::element_type`, so it wasn't applied to C++17.

Peter Dimov has confirmed that this was unintentionally lost, and that "boost::weak_ptr defines `element_type` in the same way as `shared_ptr`".

[2017-07-17 Moved to Tentatively Ready after 6 positive votes on c++std-lib.]

Proposed resolution:

This resolution is relative to [N4659](#).

1. Edit 23.11.2.3 [\[util.smartptr.weak\]](#), class template `weak_ptr` synopsis, as indicated:

```
template<class T> class weak_ptr {
public:
    using element_type = remove_extent_t<T>;
    [...]
};
```

[3024^{\(i\)}](#). variant's copies must be deleted instead of disabled via SFINAE

Section: 23.7.3.1 [\[variant.ctor\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2017-10-10 **Last modified:** 2017-10-15

Priority: Not Prioritized

View other [active issues](#) in [\[variant.ctor\]](#).

View all other [issues](#) in [\[variant.ctor\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

The specification of `variant`'s copy constructor and copy assignment operator require that those functions do not participate in overload resolution unless certain conditions are satisfied. There is no mechanism in C++ that makes it possible to prevent a copy constructor or copy assignment operator from participating in overload resolution. These functions should instead be specified to be defined as deleted unless the requisite conditions hold, as we did for the copy constructor and copy assignment operator of `optional` in LWG [2756](#).

[2017-10-11 Moved to Tentatively Ready after 5 positive votes on c++std-lib.]

Proposed resolution:

This wording is relative to [N4687](#).

1. Change 23.7.3.1 [\[variant.ctor\]](#) as indicated:

```
variant(const variant& w);
```

-6- *Effects:* If `w` holds a value, initializes the `variant` to hold the same alternative as `w` and direct-initializes the contained value with `get<j>(w)`, where `j` is `w.index()`. Otherwise, initializes the `variant` to not hold a value.

-7- *Throws:* Any exception thrown by direct-initializing any τ_i for all i .

-8- *Remarks:* This ~~function shall not participate in overload resolution~~ **constructor shall be defined as deleted** unless `is_copy_constructible_v<Ti>` is true for all i .

2. Change 23.7.3.3 [\[variant.assign\]](#) as indicated:

```
variant& operator=(const variant& rhs);
```

[...]

-4- *Postconditions*: `index() == rhs.index()`.

-5- *Remarks*: This ~~function shall not participate in overload resolution~~ operator shall be defined as deleted unless `is_copy_constructible_v<Ti> && is_copy_assignable_v<Ti>` is true for all i .